

Parte 1: React e React Native

1. b) `useState` (2 pontos)
 2. a) React Navegação (2 pontos)
 3. a) Usar o gancho `useMemo` (2 pontos)
 4. a) `width: "50%"` (5 pontos)
 5. a) Substituir Redux e gerenciar estado global sem bibliotecas externas (5 pontos)
 6. b) React é usado para web e React Native é usado para dispositivos móveis. (2 pontos)
-

Parte 2: JavaScript

7. b) "objeto" (1 pontos)
 8. b) `false` (1 pontos)
 9. a) "[objeto Objeto]" (1 pontos)
 10. d) Ambas as opções a e b estão corretas (1.5 pontos)
 11. a) `var` é escopo de função, enquanto `let` e `const` são escopos de bloco. (1.5 pontos)
 12. a) 10 (1.5 pontos)
 13. d) Operador de coalescência nula (1.5 pontos)
 14. d) Todas as opções acima (1 pontos)
-

Parte 3: TypeScript

15. a) `let x: number = 5;` (2 pontos)
 16. b) Evitar a necessidade de declaração do tipo manualmente. (2 pontos)
 17. a) `function(arg: T): T {}` (3 pontos)
 18. b) Substituir `any` com maior controle. (3 pontos)
 19. d) Tanto b quanto c estão corretas. (3 pontos)
 20. c) Erro de compilação porque `toUpperCase` não existe no tipo `number`. (2 pontos)
 21. a) `readonly` é usado em objetos e propriedades, enquanto `const` é usado para variáveis. (3 pontos)
 22. d) Ambas as opções a e c estão corretas. (3 pontos)
-

Parte 4: Lógica de Programação

23. a) 11 (4 pontos)
 24. b) `array[2]` (4 pontos)
 25. a) Dividir o array ao meio e procurar no lado correto. (3 pontos)
 26. c) 50 (2 pontos)
 27. b) 2 (2 pontos)
 28. a) `number` (2 pontos)
-

Parte 5: Clean Code e KISS

- 29. a) `return x > 10 ? "maior" : "menor";` (2 pontos)
 - 30. b) `calculateSum` (1 pontos)
 - 31. b) Código que qualquer desenvolvedor possa ler e entender rapidamente. (1 pontos)
 - 32. a) São difíceis de testar e manter. (1 pontos)
 - 33. c) Adicione muitos comentários em código claro. (1 pontos)
 - 34. a) Evitar duplicação de lógica ou dados no código. (1 pontos)
 - 35. a) Reescrever partes do código sem alterar sua funcionalidade. (1 pontos)
 - 36. b) Evitar implementar funcionalidades desnecessárias antes que sejam realmente necessárias. (1 pontos)
 - 37. a) Os comentários podem ficar desatualizados e gerar confusão. (1 pontos)
-

Parte 6: Programação Orientada a Objetos

- 38. b) Encapsulação, Herança, Polimorfismo (2 pontos)
- 39. a) A capacidade de ocultar os detalhes internos de implementação de uma classe. (2 pontos)
- 40. a) `abstract` (2 pontos)
- 41. a) `private` permite acesso apenas dentro da classe, enquanto `protected` permite acesso também em classes derivadas. (2 pontos)
- 42. a) Alterar o comportamento de um método em uma classe filha. (2 pontos)
- 43. b) "Latido" (2 pontos)
- 44. a) Sobrecarga (2 pontos)
- 45. a) Abstração (2 pontos)
- 46. c) 30 (2 pontos)
- 47. c) Uma classe que utiliza objetos de outra classe para compor seu comportamento. (2 pontos)